

# Reconhecimento de Expressões Faciais com Arquitetura AnyNet: Busca de Arquitetura Neural via Otimização com Optuna

Caio Bandeira Moreira<sup>1</sup>, Davih Asaph Tavares Alves<sup>1</sup>,  
José Marques Viana Júnior<sup>1</sup>, Maurício Miranda Celani<sup>1</sup>

<sup>1</sup>Disciplina de Visão Computacional  
Universidade Federal do Maranhão (UFMA)

{moreira.caio, davih.asaph, jmv.junior, mauricio.celani}@discente.ufma.br

**Abstract.** *This work presents the development of a convolutional neural network for facial expression recognition across 8 emotion classes using the FER+ dataset. Following the AnyNet design paradigm, the architecture is defined by parameterizable stages — number of stages, width, depth, and block type — rather than being manually fixed. Two block types were defined as candidates: PlainConv, a simplified residual block, and MBConv, the Mobile Inverted Bottleneck from EfficientNet. Architecture search was performed with Optuna using the TPE sampler, distributed across 3 machines with 30 independent trials each, totaling 90 initial trials. These trials were then evaluated to determine which batch should receive an additional 30 trials. All experiments were monitored via Weights & Biases. The best configuration found uses exclusively PlainConv blocks across 3 stages, achieving a macro F1-score of  $0.8975 \pm 0.0015$  on FER+ under 3-fold cross-validation. Generalization was evaluated on the CK+ dataset, reaching a macro F1-score of 0.4243, with stronger performance on highly expressive classes such as happy (0.96) and surprise (0.95). Furthermore, a real-time application was developed combining YOLO for face detection with the trained classifier, and Grad-CAM was applied to interpret the model's decisions.*

**Resumo.** *Este trabalho apresenta o desenvolvimento de uma rede neural convolucional para reconhecimento de expressões faciais em 8 classes de emoção utilizando o dataset FER+. Seguindo o paradigma de design AnyNet, a arquitetura é definida por estágios parametrizáveis — número de estágios, largura, profundidade e tipo de bloco — em vez de ser fixada manualmente. Dois tipos de blocos foram definidos como candidatos: PlainConv, um bloco residual simplificado, e MBConv, o Mobile Inverted Bottleneck do EfficientNet. A busca de arquitetura foi realizada com Optuna utilizando o amostrador TPE, distribuída entre 3 máquinas com 30 trials independentes cada, totalizando 90 trials iniciais, avaliando esses trials e buscando qual bateria de 30 trials deve receber mais 30. Todos os experimentos foram monitorados via Weights & Biases. A melhor configuração encontrada utiliza exclusivamente blocos PlainConv em 3 estágios, atingindo F1 macro de  $0,8975 \pm 0,0015$  no FER+ sob validação cruzada de 3 folds. A generalização foi avaliada no dataset CK+, alcançando F1 macro de 0,4243, com melhor desempenho nas classes mais expressivas como happy (0,96) e surprise (0,95). Uma aplicação em tempo real foi desenvolvida*

*utilizando YOLO para detecção de faces combinada com o classificador treinado, e Grad-CAM foi aplicado para interpretar as decisões do modelo.*

## 1. Introdução

Este trabalho tem como objetivo o desenvolvimento de uma rede neural convolucional para reconhecimento de expressões faciais em 8 classes utilizando o dataset FER+ [Barsoum et al. 2016].

A abordagem adotada segue os princípios do AnyNet [Radosavovic et al. 2020], onde a arquitetura é definida por parâmetros de design como número de estágios, largura, profundidade e tipo de bloco em vez de ser fixada manualmente. A busca pela melhor configuração é realizada pelo framework Optuna [Akiba et al. 2019], e os experimentos são monitorados via Weights & Biases [Biewald 2020].

Para avaliar a generalização do modelo, foram realizados experimentos no dataset CK+ [Lucey et al. 2010] e testes em uma aplicação em tempo real com detecção de faces via YOLO [Redmon et al. 2016]. A técnica Grad-CAM [Selvaraju et al. 2017] foi utilizada para interpretar as decisões do modelo.

## 2. Dataset

### 2.1. FER+

O FER+ é uma versão aprimorada do FER2013, com reanotação por múltiplos anotadores humanos. Cada imagem possui votos distribuídos entre 8 classes de emoção: *angry*, *contempt*, *disgust*, *fear*, *happy*, *neutral*, *sad* e *surprise*. O dataset completo possui 78293 imagens.

**Tabela 1. Distribuição de classes no FER+**

| Classe       | Amostras      |
|--------------|---------------|
| neutral      | 12.994        |
| happy        | 9.929         |
| surprise     | 9.450         |
| sad          | 9.449         |
| angry        | 9.322         |
| fear         | 9.098         |
| contempt     | 9.030         |
| disgust      | 9.021         |
| <b>Total</b> | <b>78.293</b> |

## 3. Arquitetura

### 3.1. Visão Geral

A rede proposta segue o paradigma AnyNet, sendo composta por três componentes principais: *Stem*, estágios convolucionais e *Head*.

### 3.2. Stem

O *Stem* é o estágio de processamento inicial da rede. Ele recebe a imagem de entrada com 3 canais ( $48 \times 48$  pixels) e aplica uma convolução simples de  $3 \times 3$ , seguida por *Batch Normalization* e ativação ReLU, gerando as características primárias e o volume de tensores base para os estágios subsequentes.

---

```
1 class Stem(nn.Module):
2     def __init__(self, out_ch=32):
3         super().__init__()
4         self.stem = nn.Sequential(
5             nn.Conv2d(3, out_ch, kernel_size=3, stride=1,
6                       padding=1, bias=False),
7             nn.BatchNorm2d(out_ch),
8             nn.ReLU(inplace=True),
9         )
10    def forward(self, x):
11        return self.stem(x)
```

---

Listing 1. Implementação da camada *Stem*.

### 3.3. Blocos Construtivos

Dois tipos de blocos foram definidos como candidatos na busca:

#### 3.3.1. PlainConv

Bloco residual simplificado inspirado no *BasicBlock* da ResNet [He et al. 2016], composto por uma única convolução  $3 \times 3$  seguida de Batch Normalization e ReLU, com *shortcut connection* via projeção  $1 \times 1$  quando necessário. A ideia era fazer um bloco simples que pudesse ser combinado de diferentes formas com o bloco MBConv que será falado logo a seguir.

---

```
1 class PlainConv(nn.Module):
2     def __init__(self, in_ch, out_ch, stride=1):
3         super().__init__()
4         self.conv = nn.Sequential(
5             nn.Conv2d(in_ch, out_ch, 3, stride=stride, padding
6                       =1, bias=False),
7             nn.BatchNorm2d(out_ch),
8             nn.ReLU(inplace=True),
9         )
10        # Projecao 1x1 se canais ou stride mudam
11        self.shortcut = nn.Sequential()
12        if stride != 1 or in_ch != out_ch:
13            self.shortcut = nn.Sequential(
14                nn.Conv2d(in_ch, out_ch, 1, stride=stride, bias=
15                          False),
16                nn.BatchNorm2d(out_ch),
```

```

15         )
16
17     def forward(self, x):
18         return self.conv(x) + self.shortcut(x)

```

---

**Listing 2. Estrutura do PlainConv.**

### 3.3.2. MBConv

Bloco *Mobile Inverted Bottleneck* utilizado no EfficientNet (que é uma adaptação do bloco da MobileNet) [Tan and Le 2019], composto por expansão de canais via convolução  $1 \times 1$ , convolução *depthwise*  $3 \times 3$  ou  $5 \times 5$ , bloco SE (*Squeeze-and-Excitation*) e projeção final.

---

```

1  class SEBlock(nn.Module):
2      def __init__(self, ch, ratio=4):
3          super().__init__()
4          r = max(1, ch // ratio)
5          self.se = nn.Sequential(
6              nn.AdaptiveAvgPool2d(1),
7              nn.Flatten(),
8              nn.Linear(ch, r, bias=False),
9              nn.ReLU(inplace=True),
10             nn.Linear(r, ch, bias=False),
11             nn.Sigmoid(),
12         )
13
14     def forward(self, x):
15         w = self.se(x).view(x.size(0), -1, 1, 1)
16         return x * w
17
18
19  class MBConv(nn.Module):
20      def __init__(self, in_ch, out_ch, expand_ratio=6,
21                  kernel_size=3, stride=1):
22          super().__init__()
23          mid_ch = in_ch * expand_ratio
24          padding = kernel_size // 2
25
26          self.conv = nn.Sequential(
27              # Expand
28              nn.Conv2d(in_ch, mid_ch, 1, bias=False),
29              nn.BatchNorm2d(mid_ch),
30              nn.ReLU(inplace=True),
31              # Depthwise
32              nn.Conv2d(mid_ch, mid_ch, kernel_size,
33                      stride=stride, padding=padding,
34                      groups=mid_ch, bias=False),
35              nn.BatchNorm2d(mid_ch),
36              nn.ReLU(inplace=True),

```

```

36         # SE
37         SEBlock(mid_ch),
38         # Project
39         nn.Conv2d(mid_ch, out_ch, 1, bias=False),
40         nn.BatchNorm2d(out_ch),
41     )
42
43     # Projecao 1x1 (opcao A)
44     self.shortcut = nn.Sequential()
45     if stride != 1 or in_ch != out_ch:
46         self.shortcut = nn.Sequential(
47             nn.Conv2d(in_ch, out_ch, 1, stride=stride, bias=
48                 False),
49             nn.BatchNorm2d(out_ch),
50         )
51
52     def forward(self, x):
53         return self.conv(x) + self.shortcut(x)

```

---

**Listing 3. Implementação das classes SEBlock e MBConv.**

### 3.4. Parâmetros de Design

A tabela a seguir resume o espaço de busca definido para o Optuna:

**Tabela 2. Espaço de busca da arquitetura**

| Parâmetro                | Tipo         | Intervalo            |
|--------------------------|--------------|----------------------|
| Número de estágios       | Inteiro      | [2, 4]               |
| Largura por estágio      | Inteiro      | [16, 256] (step=16)  |
| Profundidade por estágio | Inteiro      | [1, 4]               |
| Estágio de transição     | Inteiro      | [1, num_stages+1]    |
| Dropout                  | Contínuo     | [0.1, 0.5]           |
| Learning rate            | Log-uniforme | $[10^{-4}, 10^{-2}]$ |

## 4. Metodologia

### 4.1. Pré-processamento

As imagens foram convertidas para escala de cinza e redimensionadas para  $48 \times 48$  pixels

7. Durante o treinamento, foram aplicadas as seguintes técnicas de *data augmentation*:

- Espelhamento horizontal aleatório ( $p=0.5$ )
- Normalização com média e desvio padrão de 0.5

### 4.2. Treinamento

Todos os experimentos utilizam validação cruzada estratificada com  $k = 3$  folds. Os dois folds de treino reservam 10% das amostras para validação interna, utilizada pelo Optuna durante a busca. O fold restante é usado como teste.

### 4.3. Busca de Arquitetura com Optuna

O Optuna utiliza o algoritmo TPE (*Tree-structured Parzen Estimator*) para amostrar configurações do espaço de busca. Cada trial treina a arquitetura por 8 épocas nos 3 folds e retorna o F1 macro médio como métrica de otimização.

A busca foi distribuída entre 3 máquinas, cada uma máquina executou 30 trials independentes, dessa forma exploramos diferentes espaços de busca resultando em um total de 90 trials total. As configurações de cada trial são persistidas em arquivos JSON para que seja possível apenas pegar a configuração do melhor modelo, para realizar um treinamento final e salvar os pesos do modelo para ser utilizado.

Utilizamos o F1 geral (média entre os 3 folds) como parâmetro para definir qual era o melhor modelo para cada bateria de 30 trials e por fim o melhor entre os melhores de cada estudo (**Melhor modelo das 90 Trials**).

O estudo de 30 trials relacionado ao melhor modelo é, então, continuado por mais 30 trials. Foi novamente realizada a busca pelo melhor trial presente entre as 60 (**Melhor modelo das 120 Trials**).

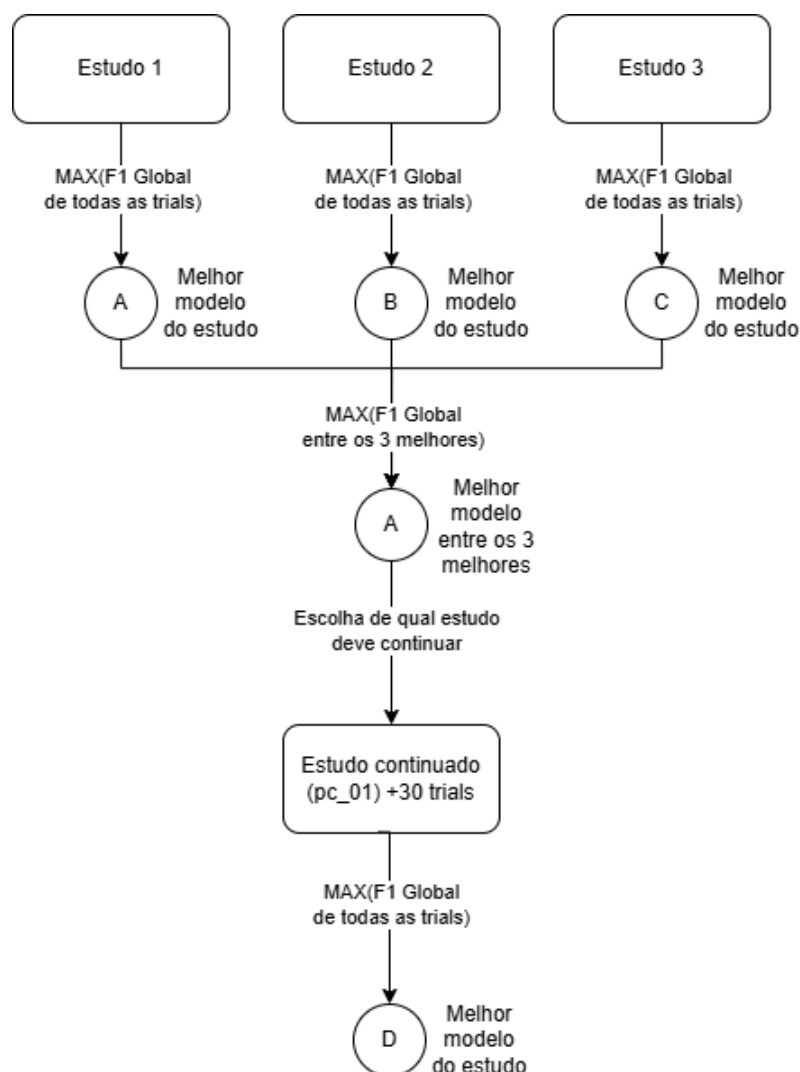


Figura 1. Caption

A figura acima ilustra como foi feito o processo de busca da arquitetura utilizada.

#### 4.3.1. Monitoramento com Weights & Biases

Todos os experimentos foram monitorados em tempo real utilizando a plataforma *Weights & Biases* (W&B) [Biewald 2020]. Cada trial do Optuna gerou uma *run* independente no projeto `fer-anyonet`, registrando métricas por época (loss de treino, acurácia e F1 macro de validação) e os hiperparâmetros da arquitetura testada (número de estágios, larguras, profundidades e estágio de transição). As runs foram organizadas em grupos por estudo, permitindo a comparação visual entre trials e entre as diferentes baterias de busca distribuída.

O W&B foi especialmente útil para identificar padrões entre os trials por exemplo para verificar quais as configurações com melhor F1 tendiam a utilizar. Mas principalmente acompanhamento do treino, em questão de estabilização de loss, f1 etc.

#### 4.4. Detecção utilizando Yolo

Para localizar os rostos nas imagens capturadas pela câmera, foi utilizado o modelo YOLOv11n-Face. Ele foi escolhido por ser uma versão mais leve da família YOLO, o que facilita sua execução em tempo real, inclusive em computadores sem uma GPU dedicada.

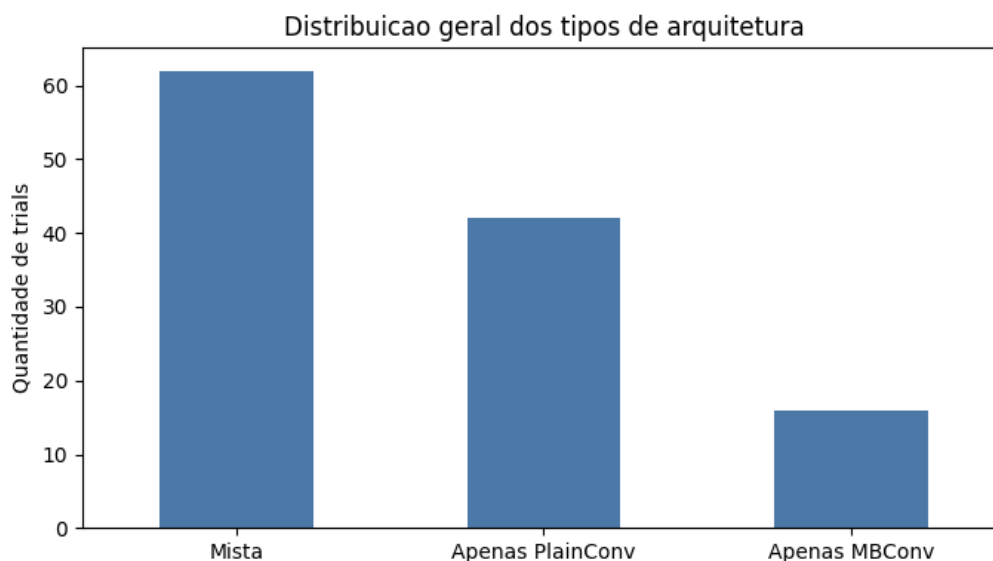
A cada quadro capturado pela webcam, o modelo analisa a imagem e identifica todos os rostos presentes, retornando a posição de cada um por meio de caixas delimitadoras, além de um valor de confiança para cada detecção. No sistema, foi adotado inicialmente um limiar de confiança de 0,50, que pode ser ajustado durante a execução.

Após a detecção, o programa percorre todos os rostos encontrados no quadro, e não apenas o primeiro. Cada face é recortada individualmente e enviada para a próxima etapa do sistema, responsável pela classificação da expressão facial.

### 5. Experimentos e Resultados

#### 5.1. Busca de Arquitetura

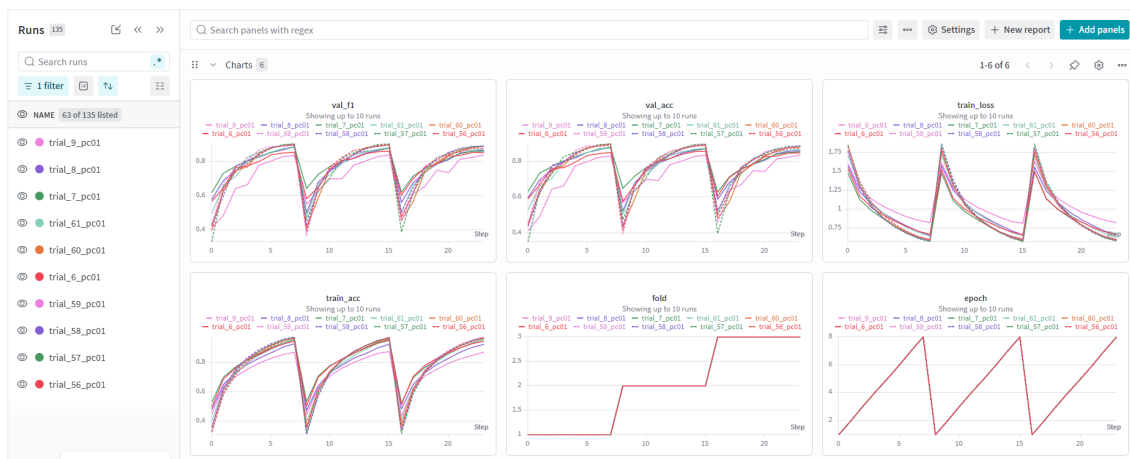
Nossa busca pela melhor arquitetura resultou em um grande número de arquiteturas mistas que utilizaram bastante arquiteturas mistas (com PlainConv e MBConv) e também um número considerável de arquiteturas que apenas usavam o bloco mais básico PlainConv. No entanto, as arquiteturas que utilizavam apenas MBConv foram minorias nos experimentos, segue a distribuição de distribuição de arquiteturas divididas em Mistas, Apenas PlainConv e Apenas MBConv:



**Figura 2. Distribuição dos tipos de arquiteturas.**

## 5.2. Acompanhamento do treino no Weights and Biases

A Figura 3 ilustra o painel do W&B durante a busca, onde é possível observar a evolução do F1 macro de validação ao longo das épocas para diferentes trials e folds.



**Figura 3. Painel do W&B durante a busca de arquitetura. Cada linha representa um trial diferente, com métricas registradas por época e por fold.**

É possível perceber que a partir de 5 épocas as métricas de F1 e acurácia começam a estabilizar mais. Isso pôde ser observado em todos as runs que foram feitas. Mostrando que as 8 épocas eram suficientes para os modelos convergirem.

## 5.3. Melhor Configuração Encontrada

Utilizando a nossa metodologia, esses foram os Modelo A e Modelo D selecionados



**Tabela 3. Configuração da melhor arquitetura encontrada (MODELO A)**

| Parâmetro            | Valor             |
|----------------------|-------------------|
| Número de estágios   | 3                 |
| Larguras             | [192, 192, 208]   |
| Profundidades        | [3, 4, 4]         |
| Tipo de bloco        | PlainConv (todos) |
| Estágio de transição | 4                 |
| Dropout              | 0.31              |
| Learning rate        | 0.00306           |
| Número de parâmetros | 3,664,584         |

**Tabela 4. Configuração da melhor arquitetura encontrada (MODELO D)**

| Parâmetro            | Valor             |
|----------------------|-------------------|
| Número de estágios   | 3                 |
| Larguras             | [144, 192, 256]   |
| Profundidades        | [4, 4, 3]         |
| Tipo de bloco        | PlainConv (todos) |
| Estágio de transição | 4                 |
| Dropout              | 0.2069            |
| Learning rate        | 0.009916          |
| Número de parâmetros | 3,557,320         |

Em ambos os casos, o Optuna selecionou modelos que utilizavam nosso PlainConv em todas as camadas. Mostrando que foi preferível uma arquitetura que utilizava blocos mais simples.

#### 5.4. Resultados no FER+

**Tabela 5. Comparação dos resultados da validação cruzada no dataset FER+**

| Fold         | Modelo A            |                     | Modelo D            |                     |
|--------------|---------------------|---------------------|---------------------|---------------------|
|              | Accuracy            | F1 Macro            | Accuracy            | F1 Macro            |
| Fold 1       | 0.8944              | 0.8992              | 0.9000              | 0.9049              |
| Fold 2       | 0.8933              | 0.8978              | 0.8952              | 0.9001              |
| Fold 3       | 0.8904              | 0.8955              | 0.8921              | 0.8974              |
| <b>Média</b> | $0.8927 \pm 0.0021$ | $0.8975 \pm 0.0015$ | $0.8958 \pm 0.0040$ | $0.9008 \pm 0.0031$ |

#### 5.5. Inferência na CK+

Após a realização da busca por uma arquitetura gerada com base na FER+, foi realizado um experimento para verificar como os modelos obtidos se saíam em um dataset

diferente, ou seja, testar a capacidade de generalização dos modelos, essa etapa foi considerada o pente fino para a escolha do modelo que seria testado na aplicação em câmera.

**Tabela 6. Distribuição de classes no dataset CK+**

| Classe       | Amostras   |
|--------------|------------|
| surprise     | 249        |
| happy        | 207        |
| disgust      | 177        |
| anger        | 135        |
| sadness      | 84         |
| fear         | 75         |
| contempt     | 54         |
| <b>Total</b> | <b>981</b> |

**Tabela 7. Comparação de desempenho entre os Modelos A e D no dataset CK+**

| Métrica                    | Modelo A      | Modelo D    |
|----------------------------|---------------|-------------|
| <b>Métricas Globais</b>    |               |             |
| Accuracy                   | <b>0.5525</b> | 0.5178      |
| F1 Macro                   | <b>0.4243</b> | 0.3888      |
| F1 Weighted                | <b>0.5880</b> | 0.5371      |
| <b>F1-Score por Classe</b> |               |             |
| anger                      | <b>0.12</b>   | 0.08        |
| contempt                   | 0.00          | 0.00        |
| disgust                    | <b>0.50</b>   | 0.26        |
| fear                       | <b>0.12</b>   | 0.00        |
| happy                      | <b>0.96</b>   | 0.94        |
| sadness                    | 0.31          | <b>0.50</b> |
| surprise                   | <b>0.95</b>   | 0.93        |

É possível perceber que em uma tarefa de generalização para um dataset que em teoria teria imagens semelhantes, não se saiu muito bem, no CK+ uma única imagem é repetida três vezes no dataset, talvez essa seja uma limitação do dataset mesmo, pois se você errar uma imagem, você erraria três pois as outras duas são iguais, e acreditamos que isso trouxe os nossos resultados um pouco pra baixo.

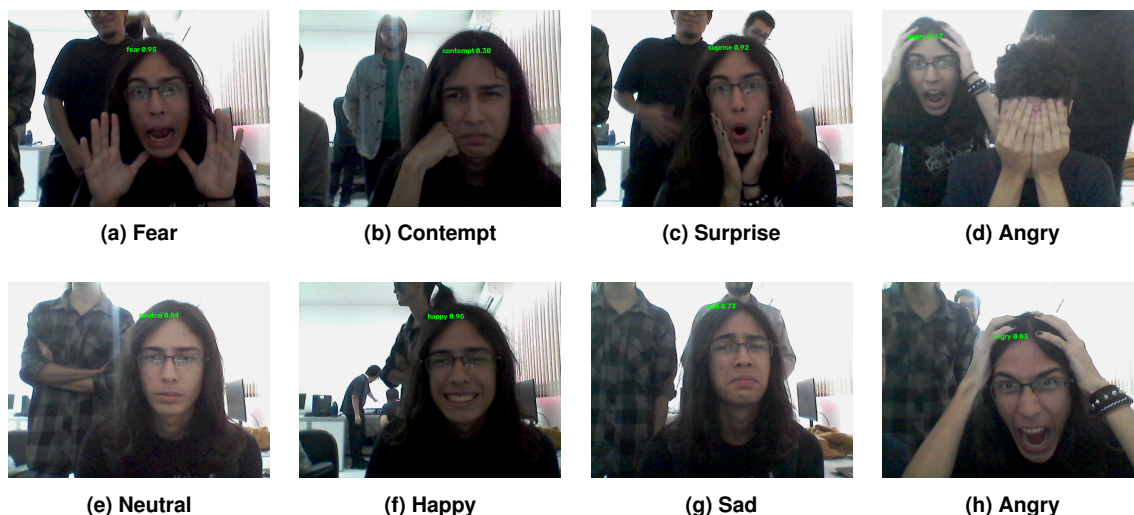
No entanto, ainda utilizamos o que se saiu melhor na generalização, na Tabela acima ilustra que o Modelo A se saiu melhor em detectar emoções de maneira mais ampla e por isso foi o escolhido em vez do Modelo D para os testes na câmera.

## 5.6. Experimentos da aplicação de câmera

Utilizando o Modelo A, foram realizados testes por meio de um .py que realizava a detecção do rosto dos indivíduos presentes no frame com a utilização da Yolo, realizava as

devidas transformações no recorte do frame e então a inferencia no modelo.

As figuras abaixo apresentam exemplos de um experimento inicial realizado.



**Figura 4. Exemplos da utilização da câmera representando 7 das 8 classes de expressões faciais presentes na FER+.**

A Figura acima demonstra a capacidade do modelo A de detectar algumas das classes presentes no FER+. Vale mencionar que o modelo ainda apresenta uma certa necessidade de realizar expressões exageradas ou caricatas. Além disso, a emoção *disgust* quase não foi percebida durante os experimentos realizados com a câmera, demonstrando que essa classe não está sendo bem detectada, não sendo possível então apresentar uma figura de exemplo desta classe.

## 6. Explicabilidade

Para interpretar as decisões do modelo, aplicou-se a técnica Grad-CAM (*Gradient-weighted Class Activation Mapping*) [Selvaraju et al. 2017] na última camada convolucional do último estágio. Os mapas de calor indicam quais regiões da face o modelo considera mais relevantes para cada predição, tons quentes (vermelho/amarelo) indicam alta ativação, tons frios (azul) indicam baixa ativação.

Foram gerados dois conjuntos de visualizações: exemplos de acertos (Verdadeiros Positivos) e exemplos de erros de classificação por classe.

### Mapas Grad-CAM representativos por classe

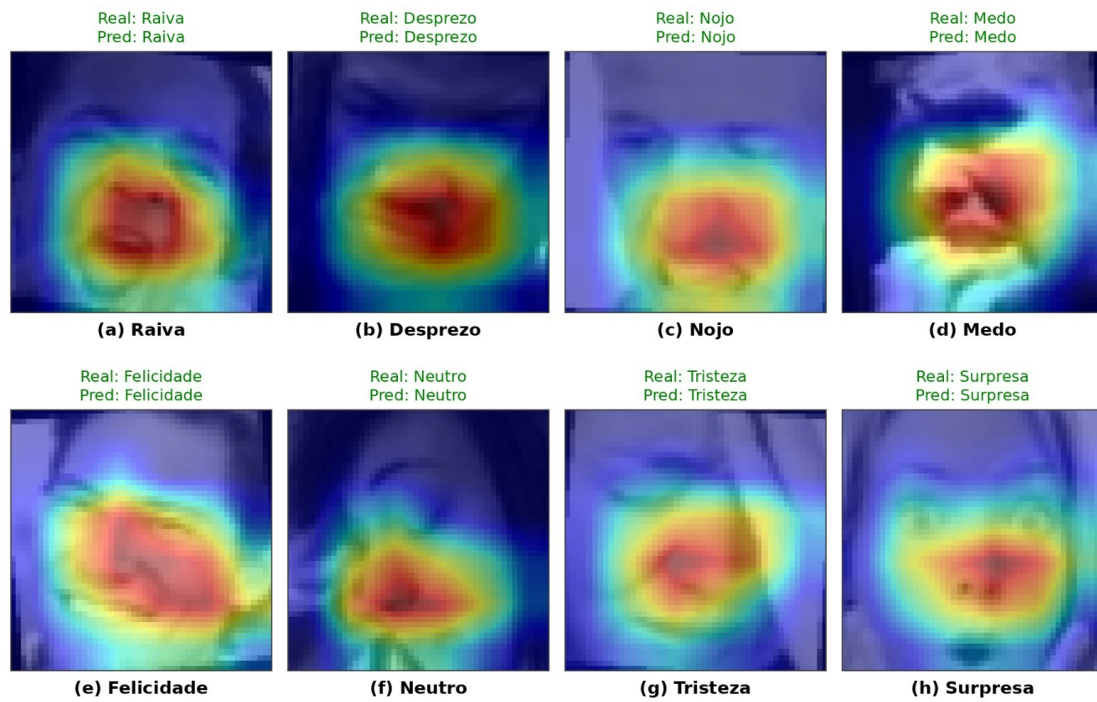


Figura 5. Mapas Grad-CAM de acertos.

Mapas Grad-CAM representativos por classe, exemplos onde o modelo classificou corretamente. O modelo tende a focar na região da boca e bochechas para a maioria das expressões.

### Exemplos de erros de classificação por classe

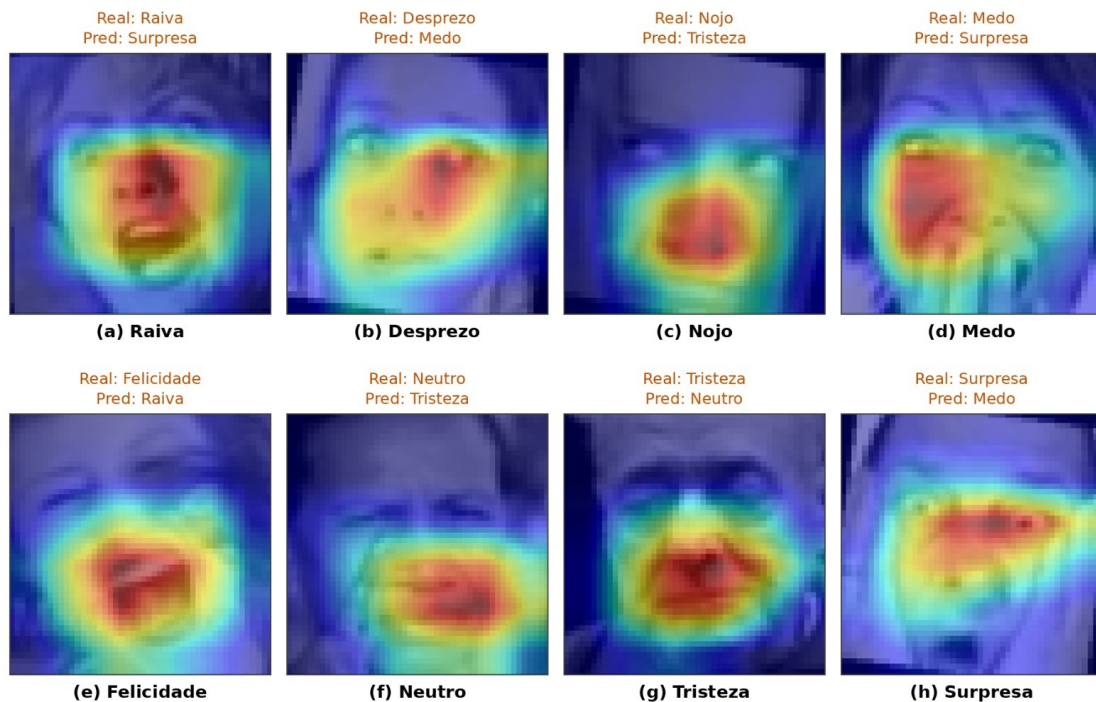


Figura 6. Mapas Grad-CAM de erros.

É possível observar que mesmo nos erros o modelo ativa regiões facialmente relevantes, sugerindo que as confusões ocorrem por similaridade visual entre classes e não por ativação de regiões irrelevantes.

## 7. Limitações

Inicialmente, a ideia era utilizar o dataset CK+, antes de ser percebido que o mesmo apresentava uma quantidade de amostras bem inferior ao FER+, por conta dessa ideia inicial, implementamos uma lógica transformações baseadas em imagens  $48 \times 48$  em escala de cinza.

Mesmo após a mudança para utilizar FER+, o resize de  $48 \times 48$  acabou sendo mantido, fato só percebido após a realização do multi-estudo. Por conta disso, o modelo foi treinado com imagens que possuem bem menos informação do que as  $112 \times 112$  originais do FER+.

Outra limitação presente no modelo utilizado é a incapacidade de detectar certas classes, *disgust*, *contempt* e *fear* são classes que muito raramente apareceram durante os experimentos com a câmera, demonstrando que há classes que o nosso modelo não foi capaz de aprender a reconhecer.

Além disso, analisando o gradcam é possível perceber que o modelo está se baseando fortemente na região da boca, bochechas e nariz. Este fato no entanto não é o esperado quando estamos analisando expressões faciais, o esperado antes da realização do treinamento do modelo era que outras regiões do rosto interferissem na classificação do sentimento, como é o caso das sombrancelhas.

## 8. Conclusão

Este trabalho apresentou o desenvolvimento de uma rede neural convolucional para reconhecimento de expressões faciais seguindo o paradigma AnyNet, onde a arquitetura é tratada como um espaço de busca otimizado pelo Optuna. Ao longo de 120 trials distribuídos entre 3 máquinas, o processo de busca convergiu para configurações que utilizam exclusivamente blocos PlainConv, sugerindo que, para imagens de  $48 \times 48$  pixels, blocos mais simples com maior largura de canais superam blocos mais sofisticados como o MBConv.

O Modelo A, selecionado como melhor entre os estudos, atingiu F1 macro de  $0,8975 \pm 0,0015$  no FER+ sob validação cruzada de 3 folds, utilizando uma arquitetura com aproximadamente 3,6 milhões de parâmetros.

No experimento de generalização no CK+ foi observado uma queda de desempenho, (F1 macro de 0,4242), mostrando que há grande diferença entre os dados de ambos os dataset. O modelo teve bons resultados nas classes de happy (F1=0.96) e surprise (F1=0.95), e teve dificuldades em classes mais sutis como contempt e fear.

Os testes na aplicação em tempo real com YOLO confirmaram o bom funcionamento do modelo, com detecção da face satisfatória, e uma boa sensibilidade entre algumas classes como happy e neutral, happy e surprise.

## Referências

- Akiba, T., Sano, S., Yanase, T., Ohta, T., and Koyama, M. (2019). Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 2623–2631. ACM.
- Barsoum, E., Zhang, C., Ferrer, C. C., and Zhang, Z. (2016). Training deep networks for facial expression recognition with crowd-sourced label distribution. In *Proceedings of the 18th ACM International Conference on Multimodal Interaction*, pages 279–283. ACM.
- Biewald, L. (2020). Experiment tracking with weights and biases. Software available from wandb.com.
- He, K., Zhang, X., Ren, S., and Sun, J. (2016). Deep Residual Learning for Image Recognition. In *Proceedings of 2016 IEEE Conference on Computer Vision and Pattern Recognition*, CVPR '16, pages 770–778. IEEE.
- Lucey, P., Cohn, J. F., Kanade, T., Saragih, J. M., Ambadar, Z., and Matthews, I. (2010). The extended cohn-kanade dataset (ck+): A complete dataset for action unit and emotion-specified expression. In *2010 IEEE Computer Society Conference on Computer Vision and Pattern Recognition - Workshops*, pages 94–101. IEEE.
- Radosavovic, I., Kosaraju, R. P., Girshick, R., He, K., and Dollár, P. (2020). Designing network design spaces. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 10428–10436.
- Redmon, J., Divvala, S., Girshick, R., and Farhadi, A. (2016). You only look once: Unified, real-time object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 779–788.

- Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., and Batra, D. (2017). Grad-cam: Visual explanations from deep networks via gradient-based localization. In *Proceedings of the IEEE International Conference on Computer Vision*, pages 618–626.
- Tan, M. and Le, Q. V. (2019). Efficientnet: Rethinking model scaling for convolutional neural networks. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 6105–6114. PMLR.